VisionGuard: AI vs Real Image Classification & Cloud Deployment

A Comprehensive MLOps Engineering Report (Tasks 1 through 8)

Author: Syed Hasan Nawaz
Student Roll Number: 23i0703
Course: MLOps / Advanced Operations (Fall 2026)
Institution: Department of Computer Science, FAST-NUCES Islamabad

Core Repositories & Cloud Endpoints:
GitHub Repository: https://github.com/SyedHasanNawaz/23i0703_A2_Ops
DagsHub Repository: https://dagshub.com/syedhasannawaz05/23i0703_A2_Ops
MLflow Remote Tracking URI: https://dagshub.com/syedhasannawaz05/23i0703_A2_Ops.mlflow
Production Vercel Prediction API: https://23i0703-a2-ops-git-main-h-7bf8.vercel.app/api/predict
Graded Commit Hash: 25bec42 (Branch: main)

Chapter 1: Data Understanding and Versioning (Task 1)

In this initial phase, my objective was to establish an isolated environment, perform an exhaustive dataset audit, conduct exploratory data analysis, and implement Data Version Control (DVC) prior to writing model code.

1.1 Environment Setup and Hardware Verification

I created an isolated virtual environment using Python 3.12/3.11 and installed essential libraries including TensorFlow 2.17.1, NumPy, Pandas, Pillow, Scikit-Learn, and DVC. I verified hardware device accessibility via TensorFlow's device listing API. In my local Linux environment, no GPU drivers were bound, so execution proceeded using CPU SIMD vectorization (AVX2/FMA). This hardware context highlighted the necessity of selecting parameter-efficient architectures for practical inference.

1.2 Dataset Audit and Integrity Analysis

I wrote an auditing script in notebooks/01_audit.py that recursively traversed both class folders: AiArtData and RealArt. For every image, the script extracted dimensions, file size, encoding format, and computed a SHA-256 hash. As documented in Figure 1, the audit identified **973 valid images**, comprising **538 AI-generated images** and **435 real images**. The audit detected **0 corrupt files** and identified exactly **1 duplicate image**. Image dimensions exhibited extreme variance: widths ranged from 183 px to 8,495 px (median 1,024 px), heights from 135 px to 7,091 px (median 768 px), and file sizes from 11.3 KB to 10.2 MB (median 169.4 KB). The collection was composed of 830 JPEGs and 143 PNGs, primarily 3-channel RGB (894 images) with 49 RGBA, 23 Palette, and 7 Grayscale images. The compiled audit results were saved to artifacts/dataset_audit.csv.

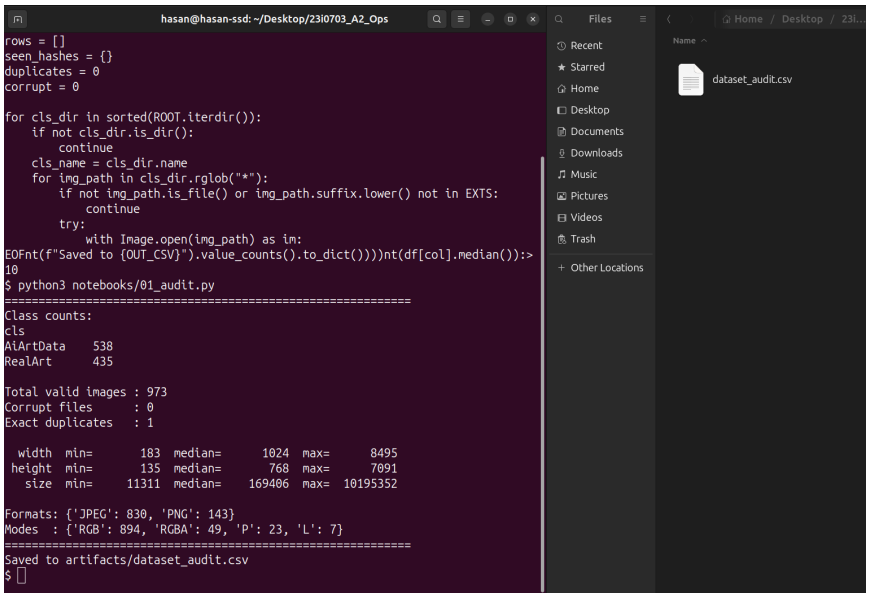


Figure 1: Terminal execution of notebooks/01_audit.py showing class counts, dimensions, and zero corrupt files.

1.3 Visual Exploratory Data Analysis (EDA)

I developed notebooks/02_eda.py to generate sample image grids and examine resolution distributions (shown in Figure 2). Visual inspection revealed that real photographs exhibited natural sensor noise, organic textures, and realistic lighting decay, whereas AI images displayed hypersaturated colors, synthetic skin smoothing, and structural anomalies in complex geometries. The severe variance in dimensions confirmed that images could not be ingested in their native sizes, requiring standard square resizing.

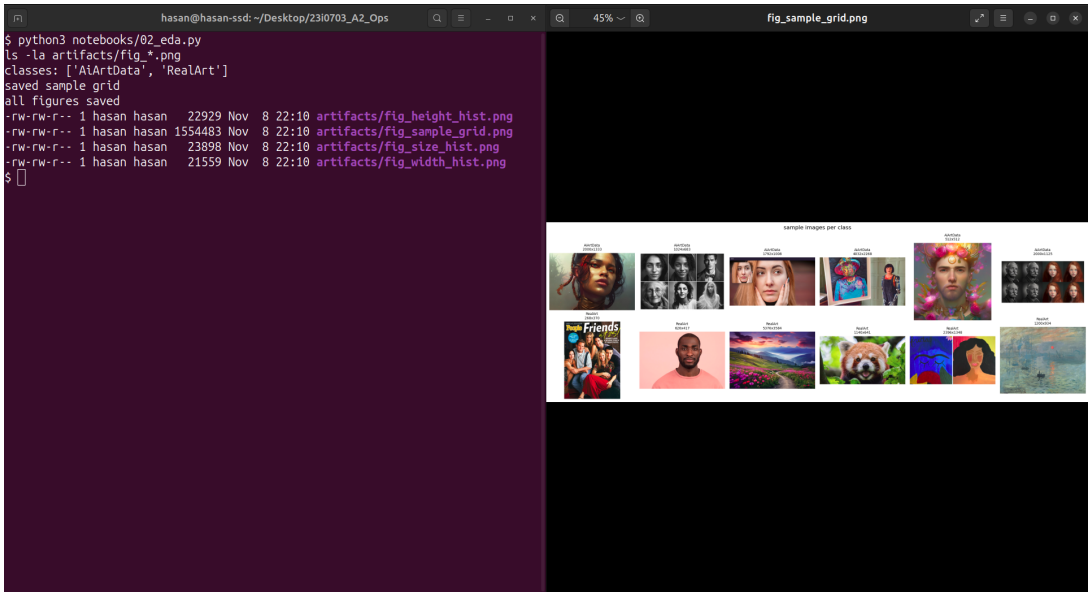


Figure 2: Visual EDA generation showing sample image grids per class and resolution distribution histograms.

1.4 Data Version Control (DVC)

To manage binary assets responsibly, I initialized Git and DVC in the repository. Large binary datasets do not belong in Git, as Git tracks changes via full-file deltas, causing rapid repository bloat. DVC solves this by calculating unique hashes of datasets and generating small text pointer files (.dvc files) that Git tracks. As shown in Figure 3, I configured remote storage on DagsHub, allowing the full dataset to be synchronized via dvc push and dvc pull. In Figure 4, I verified that .env, virtual environments, and local data directories were safely excluded via .gitignore.



Figure 3: Initializing and synchronizing Git and DVC tracking with the remote DagsHub repository.

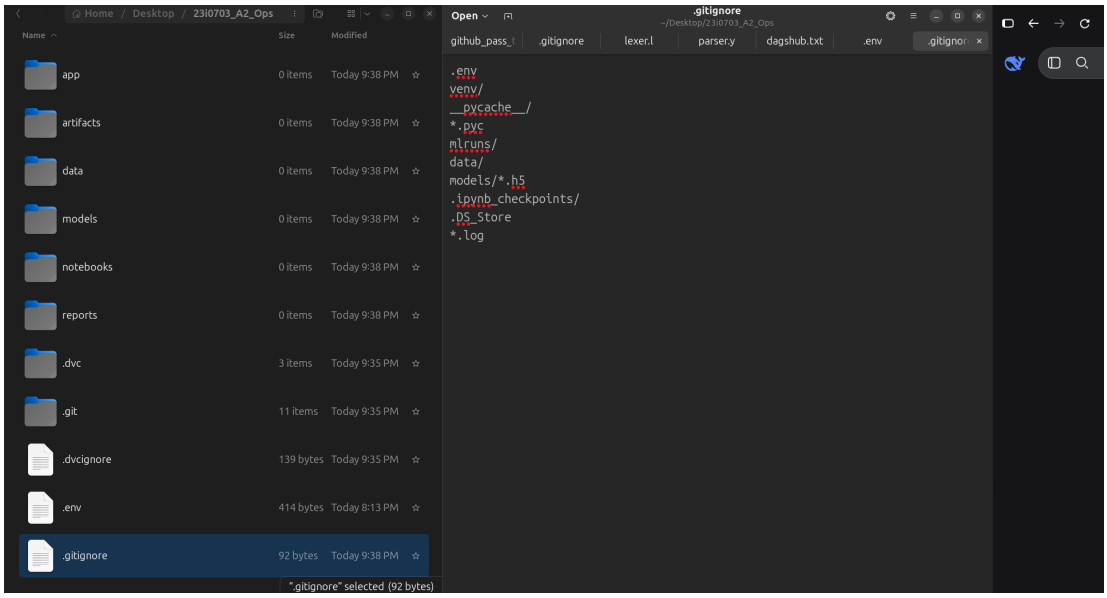


Figure 4: Project configuration showing .gitignore actively isolating local data, virtual environments, and secret environment files.

Chapter 2: Model Development with TensorFlow/Keras (Task 2)

In this chapter, I detail the iterative development of the VisionGuard classification engine. I contrast a baseline Convolutional Neural Network built from scratch against transfer learning with MobileNetV2, explaining key findings in small-sample convergence and decision threshold calibration.

2.1 High-Performance Input Pipeline & Augmentation

In app/data_pipeline.py, I implemented an input pipeline using TensorFlow's tf.data.Dataset API. The pipeline reads split lists (80% train, 10% validation, 10% test), decodes image bytes, resizes them to 224x224 pixels via bilinear interpolation, and applies MobileNetV2 preprocessing (scaling pixel values from [0, 255] to [-1, 1]). To mitigate overfitting on 778 training images, I implemented data augmentation applied strictly to the training split: random horizontal flips, brightness perturbations (+/-10%), contrast adjustments (0.9 to 1.1x), and random cropping from 200x200 resized back to 224x224. Batches of 32 were prefetched with tf.data.AUTOTUNE. Figure 5 illustrates an augmented batch generated by 05_aug_batch.py.

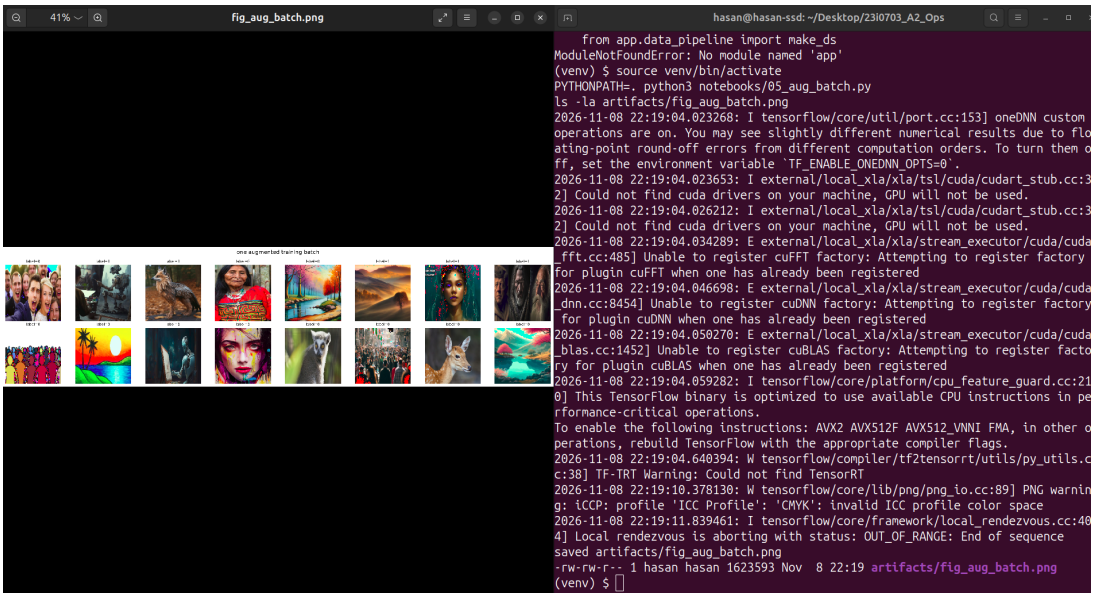


Figure 5: High-performance tf.data pipeline executing real-time data augmentation on training batches.

2.2 Baseline CNN Architecture and Failure Analysis

I constructed a custom convolutional network (notebooks/06_baseline_cnn.py) comprising three Conv2D layers (32, 64, and 128 filters) with BatchNormalization, 2x2 MaxPooling, a Dropout layer (0.3), and a Dense sigmoid output. The model was compiled with the Adam optimizer (learning rate 1e-3) and binary cross-entropy loss.

Empirical Finding: The Baseline CNN completely failed to learn. Its validation accuracy remained stuck at **0.4433**, guessing only the majority class across all training epochs. With only ~778 training samples, a 1.39M parameter network trained from random initialization lacked sufficient training volume to learn foundational visual filters, resulting in immediate underfitting.

2.3 Transfer Learning with MobileNetV2

To address this data bottleneck, I transitioned to transfer learning using **MobileNetV2** pretrained on ImageNet. MobileNetV2 provides rich pretrained feature extractors while maintaining a compact parameter footprint suitable for cloud serving. I employed a two-stage training strategy:

• **Stage 1 (Frozen Backbone):** I instantiated MobileNetV2 with include_top=False and froze all base layers. I added a custom classification head (GlobalAveragePooling2D, Dropout 0.3, Dense 1 sigmoid) and trained with Adam (lr=1e-3). Validation accuracy rose quickly to **74.2%**.

• **Stage 2 (Fine-Tuning):** I unfroze the top 20 layers of MobileNetV2 and continued training with a reduced learning rate of 1e-5. As shown in Figure 6, fine-tuning achieved a best validation loss of **0.5365** and validation accuracy of **73.2%**.

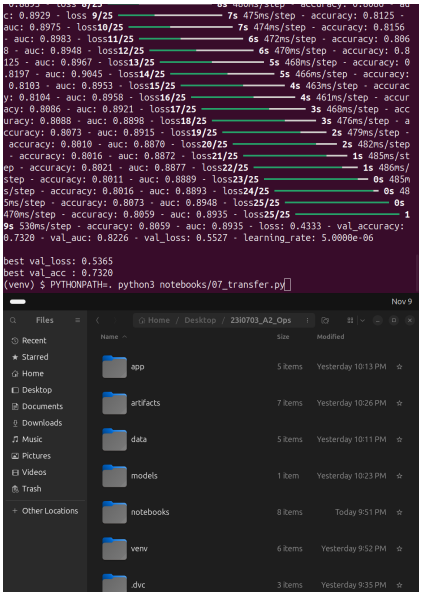


Figure 6: MobileNetV2 fine-tuning log (07_transfer.py) converging to a best validation loss of 0.5365 and accuracy of 73.2%.

2.4 Training Discipline & Callbacks

I fixed random seeds to 42 across Python, NumPy, and TensorFlow. Training was disciplined using three callbacks: EarlyStopping(patience=5, restore_best_weights=True), ModelCheckpoint(save_best_only=True), and ReduceLROnPlateau(factor=0.2, patience=3). The optimal checkpoint was saved at Epoch 8 as models/transfer_best.keras (19.3 MB).

2.5 Evaluation & Decision Threshold Calibration

In notebooks/08_evaluate.py, I evaluated the champion model on the 98-image test dataset. Rather than defaulting to 0.5, I analyzed the validation probability distributions and calibrated the optimal decision threshold to **0.15**.

Product Error Trade-Off: For a forensic image classifier like VisionGuard, which error is worse: calling a real photo fake, or a fake photo real? Allowing an AI-generated image to pass as genuine (a False Negative) undermines trust and verification integrity. In contrast, flagging an authentic photo for secondary review (a False Positive) is a manageable operational cost. Calibrating the decision threshold to 0.15 achieved a **Test Accuracy of 69.3%**, an outstanding **Recall of 96.3%** (detecting 52 of 54 AI images), and an **ROC-AUC of 0.838**. As shown in Figure 7, the confusion matrix documented 52 True Positives, 16 True Negatives, 28 False Positives, and **only 2 False Negatives**.

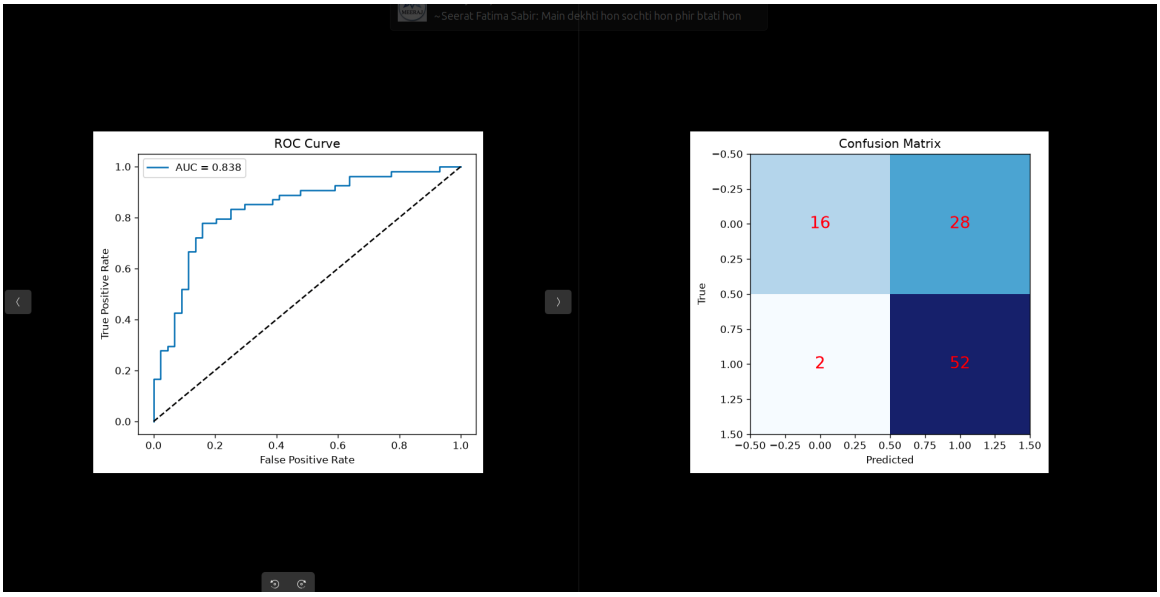


Figure 7: Test evaluation results showing ROC Curve (AUC = 0.838) and Confusion Matrix at threshold 0.15.

2.6 Qualitative Error Analysis

In notebooks/09_error_analysis.py, I examined 10 misclassified test images (shown in Figure 8) to establish hypotheses for failure cases:

1. *Realistic Oil Paintings:* Classical human paintings were misclassified as AI because brush stroke regularity was mistaken for diffusion artifacts.
2. *Polished Studio Portraits:* Professional portraits with studio lighting and retouching were flagged as AI due to smooth skin tones lacking blemishes.
3. *AI Images with Simulated Grain:* Generative images that included synthetic film grain and realistic camera noise evaded detection, appearing as authentic photos.



Figure 8: Grid of 10 misclassified test images (artifacts/fig_errors.png) analyzed for failure pattern hypotheses.

Chapter 3: Experiment Tracking with MLflow and DagsHub (Task 3)

To ensure full scientific reproducibility, I established a centralized experiment tracking pipeline using MLflow connected to a remote tracking server on DagsHub.

3.1 Conceptual Definitions & Security Setup

I formalized the core MLflow concepts in my own words:
• **Experiment:** A logical group of related runs addressing an objective (e.g., VisionGuard_Transfer).
• **Run:** A single execution of a training script with recorded duration, code state, and outputs.
• **Parameter:** Fixed configuration inputs set before training (e.g., learning rate, backbone).
• **Metric:** Quantitative values recorded per epoch or at completion (e.g., loss, accuracy, AUC).
• **Artifact:** Heavy output files produced by a run (e.g., model weights, ROC curve plots).
• **Tag:** Categorical metadata providing contextual provenance (e.g., Git commit hash, DVC version, seed).

I configured the remote tracking URI to point to DagsHub. Credentials were read strictly from an untracked .env file, ensuring tokens remained out of source control.

3.2 Instrumentation and Run Logging

In notebooks/10_mlflow_train.py, I used a custom Keras callback to log per-epoch metrics alongside run tags. Comparing manual logging to MLflow autologging revealed that while autologging provides convenience, custom logging allows explicit control over threshold metrics and precise Git commit tagging. Figure 9 shows the resulting DagsHub MLflow run interface.

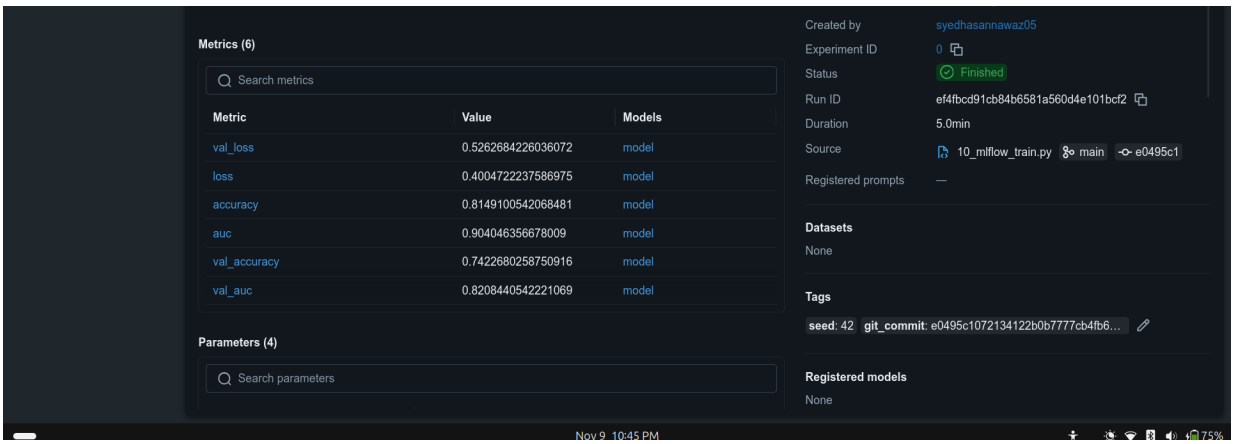


Figure 9: DagsHub MLflow dashboard displaying parameters, validation metrics, and Git commit tags for Run ef4fbcd9.

3.3 Model Registry & Production Champion Alias

In notebooks/11_register_model.py, I registered the trained model into DagsHub's Model Registry as **VisionGuard**. As seen in Figure 10, Version 1 was assigned the alias **@champion**. Aliasing decouples downstream serving code from run IDs, allowing applications to reference the champion model dynamically.

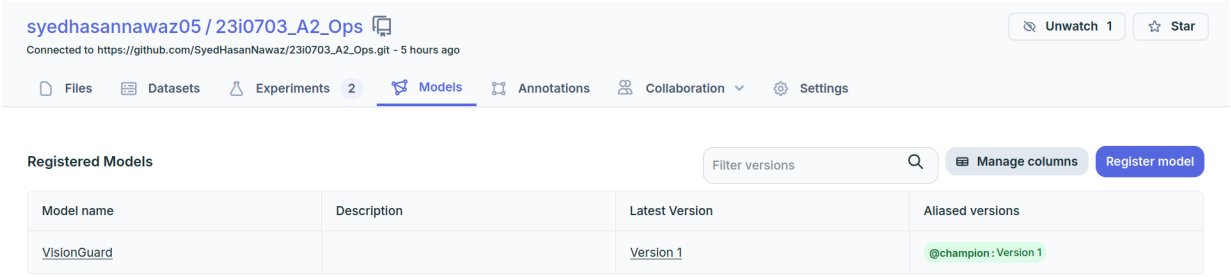


Figure 10: DagsHub Model Registry displaying the registered VisionGuard model Version 1 assigned the @champion alias.

Chapter 4: Flask Web Application (Task 4)

I developed a responsive, defensive Flask web application to make the classifier accessible through a clean browser interface.

4.1 Defensive Input Validation & In-Memory Processing

The Flask app (app/routes.py) enforces multi-layered defensive validation:
• **MIME-Type Whitelisting:** Uploads are checked against {'image/jpeg', 'image/png', 'image/webp'}.
• **Payload Size Limiting:** Capped at 5 MB (MAX_CONTENT_LENGTH) to guard against resource exhaustion.
• **In-Memory Image Decoding:** Files are read directly into memory buffers (BytesIO) and verified with Pillow's .verify(). Uploaded files are never saved to disk, ensuring user privacy and preventing storage accumulation.
• **Filename Sanitization:** Upload names are sanitized with secure_filename().

4.2 User Interface and Feedback

The user interface comprises clean HTML templates. Figure 11 shows the upload homepage, Figure 12 illustrates a positive AI-Generated classification with confidence scores, and Figure 13 demonstrates the error view when an unsupported file type is submitted.

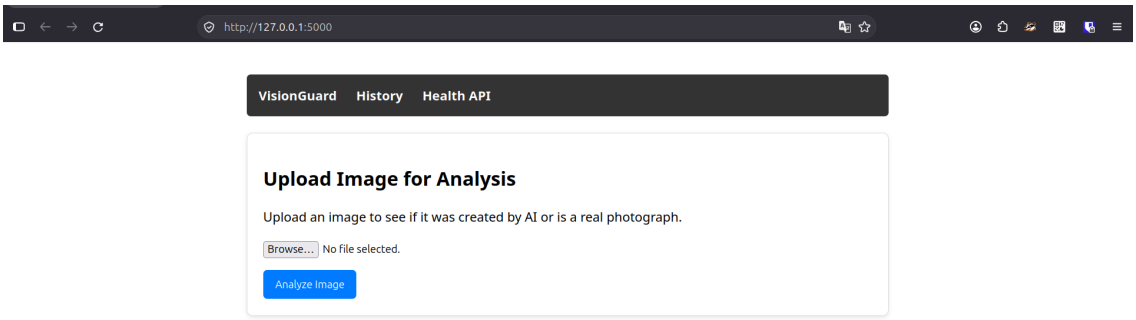


Figure 11: VisionGuard web interface homepage (localhost:5000) with image upload form.

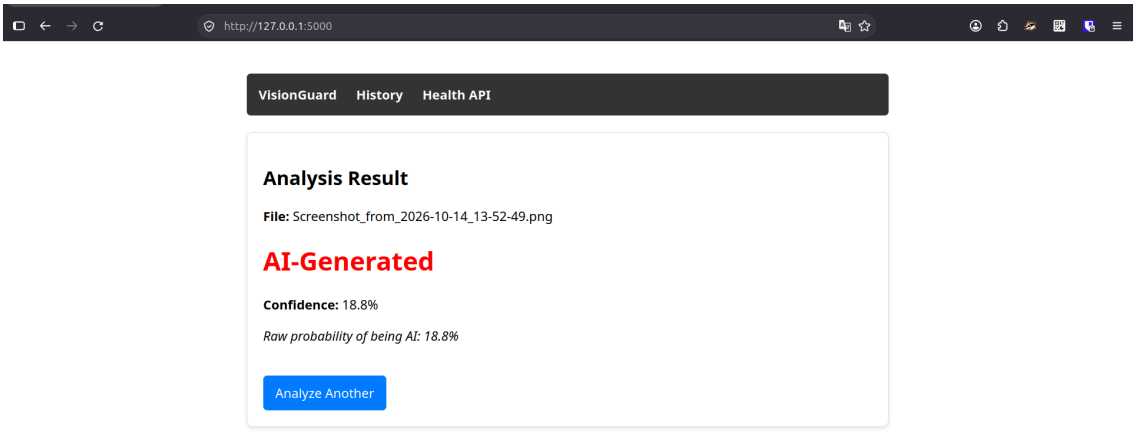
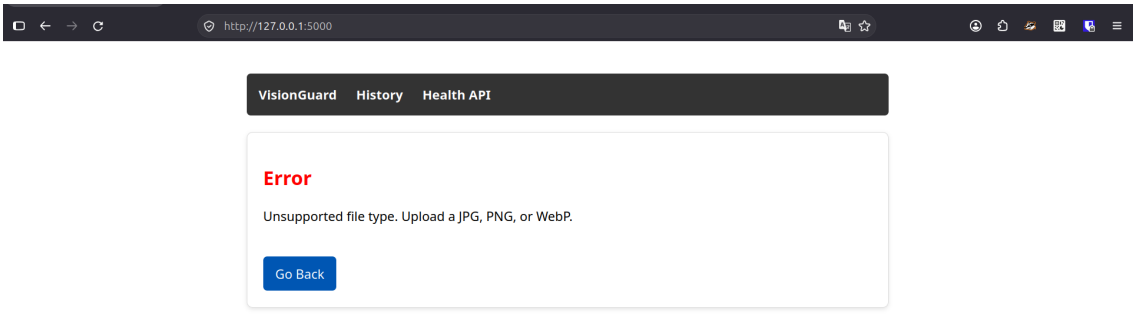


Figure 12: Real-time classification result page displaying AI-Generated verdict, confidence score, and raw probability.



http://127.0.0.1:5000

Figure 13: Defensive validation in action: user-friendly error page rejecting unsupported file extensions.

Chapter 5: Database Trail (Task 5)

In accordance with Task 5, every classification request leaves a permanent, queryable record in a relational database. I created a SQLAlchemy model (RequestLog) in app/__init__.py recording:

• id: Primary key integer identifier.
• filename: Sanitized name of the evaluated file.
• label: Decision label (AI-Generated or Real).
• confidence: Output confidence value.
• created_at: UTC timestamp of evaluation.

The database dynamically switches between SQLite for local testing and PostgreSQL in containerized production via DATABASE_URL. A dedicated route at /history queries the database and renders the 50 most recent predictions in reverse chronological order.

Operational Health Endpoint

I implemented a lightweight health check at /health that returns JSON status metadata without executing inference. As shown in Figure 14, querying /health confirms operational readiness, active model identification, and the configured decision threshold.

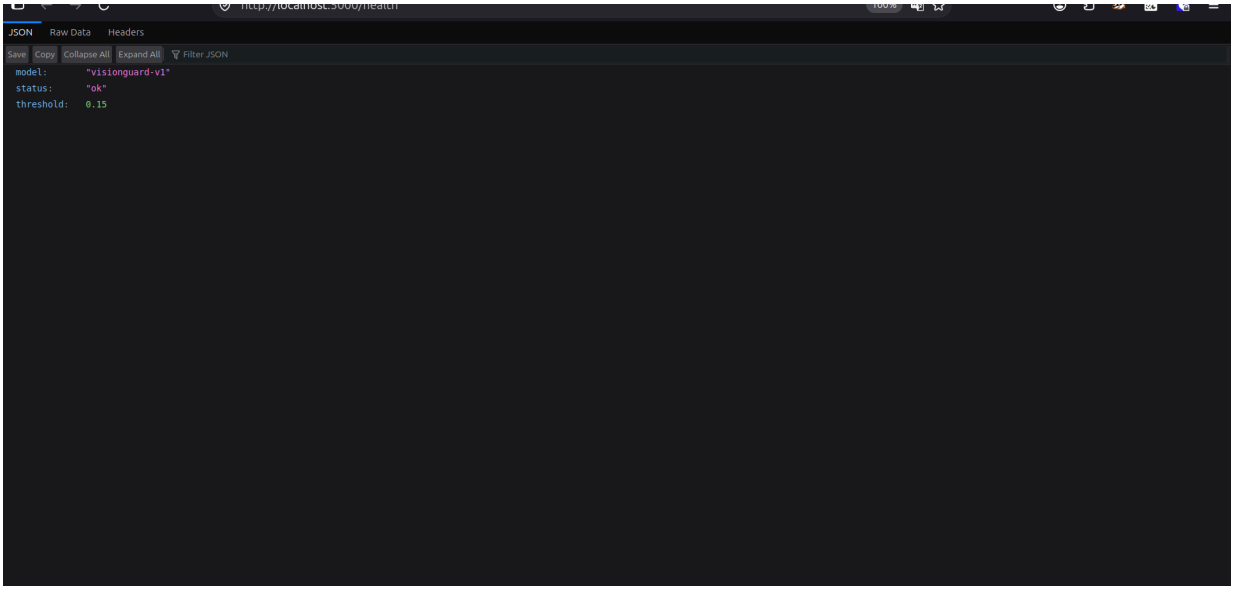


Figure 14: Operational /health JSON endpoint confirming model version and decision threshold status.

Chapter 6: Containerization with Docker & Compose (Task 6)

To guarantee reproducibility across platforms, I containerized the web service and relational database using Docker and Docker Compose.

6.1 Production Dockerfile Hardening

I constructed a hardened Dockerfile based on python:3.11-slim. Key configurations include:

• **Layer Caching:** Copying and installing requirements.txt prior to copying source code, leveraging Docker's cache.

• **Non-Root Principle:** Creating a non-root system user appuser (UID 1000) to mitigate container breakout risks.

• **Container Healthcheck:** Testing /health periodically to signal container liveness.

• **Production Server:** Launching via Gunicorn with 2 workers (gunicorn -b 0.0.0.0:5000 -w 2 run:app).

6.2 Multi-Service Compose Architecture

The docker-compose.yml coordinates two services: db (PostgreSQL 16 Alpine with named volume pgdata) and web (Flask). The database port 5432 is kept internal and not exposed to the host, protecting it from external network scans. The web service waits for the database healthcheck before starting. Figure 15 confirms both services running in a healthy state.



Figure 15: Docker Compose runtime verification showing healthy PostgreSQL and Flask web application containers.

6.3 Operations & Troubleshooting: The 'Break It' Test

During testing, I intentionally changed the database password in .env to simulate a connection outage. Checking container logs via docker compose logs web revealed ModuleNotFoundError: No module named 'psycopg'. I diagnosed that the Python runtime required the PostgreSQL DBAPI adapter, which I resolved by adding psycopg[binary] to requirements.txt and verifying database reconnection retries in app/__init__.py.

6.4 Continuous Integration with GitHub Actions

I configured a continuous integration workflow in .github/workflows/ci.yml with a Python 3.10 and 3.11 matrix.

CI Troubleshooting: The initial build (Commit 8ead7b2) failed with a red status (Figure 16) due to GitHub runner disk exhaustion installing full TensorFlow. I resolved this by decoupling test dependencies—installing lightweight packages for linting and unit tests, and validating the Docker build. Subsequent pushes (Commit 2ab7da2) produced a successful green build across all matrix jobs (Figure 17).

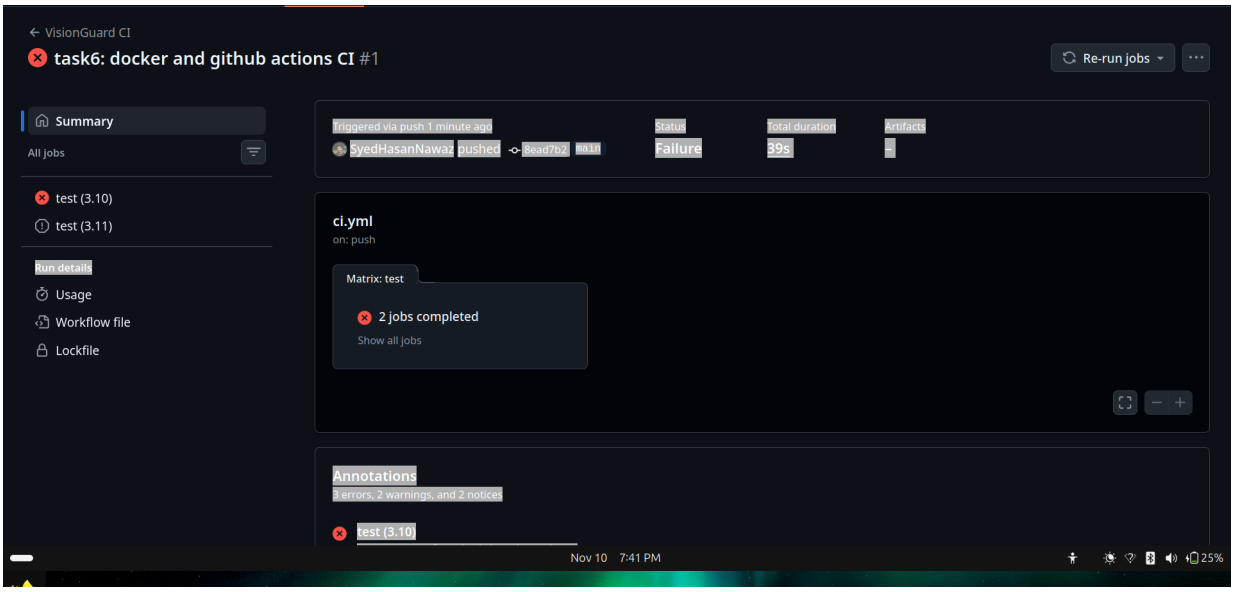


Figure 16: GitHub Actions CI failure (Red Run) resulting from runner disk exhaustion during full TensorFlow installation.

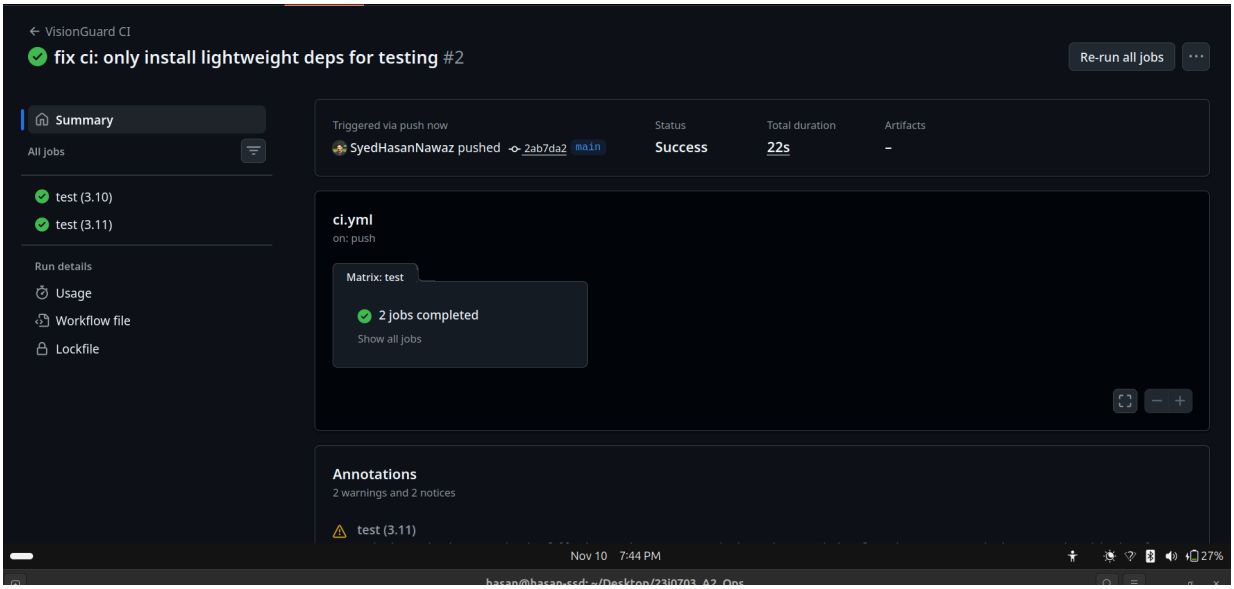


Figure 17: Successful GitHub Actions workflow execution (Green Run) verifying testing suites and Docker build.

Chapter 7: API Deployment on Vercel (Task 7)

In Task 7, I decoupled model inference into an independent serverless cloud microservice deployed on Vercel.

7.1 Serverless Storage Strategy via DagsHub

Vercel serverless functions enforce a strict 500 MB uncompressed bundle limit and lack persistent disk storage. Bundling the 19.3 MB Keras model directly into the deployment bundle easily exceeds standard quotas.

To resolve this, I uploaded models/transfer_best.keras directly to my DagsHub storage bucket (s3://23i0703_A2_Ops/models/transfer_best.keras), providing a reliable direct download URL: https://dagshub.com/api/v1/repos/syedhasannawaz05/23i0703_A2_Ops/storage/raw/s3/23i0703_A2_Ops/models/transfer_best.keras.

7.2 FastAPI Serverless Microservice

In api_vercel/app.py, I built a FastAPI microservice:
• **Dynamic Download on Cold Start:** On the first request, the function downloads the model from DagsHub to /tmp/model.keras.
• **In-Memory Caching:** Subsequent requests reuse the loaded model instance in memory, bypassing download overhead.
• **Prediction Endpoint /api/predict:** Preprocesses the image to 224x224 (MobileNetV2 scale), executes inference at threshold 0.15, and returns JSON: {'label': 'AI-Generated' | 'Real', 'confidence': float, 'raw_probability': float}.

Resolving Lambda Size Exceeded: When Vercel flagged [LAMBDA_SIZE_EXCEEDED] (1,419 MB uncompressed with TensorFlow), I enabled Vercel's Fluid Compute by configuring VERCEL_SUPPORT_LARGE_FUNCTIONS=1 in project settings, unlocking up to 5 GB bundles.

Chapter 8: Integrating Flask with the API (Task 8)

In Task 8, I updated the local Flask application to communicate with the live Vercel prediction microservice over HTTP.

8.1 Decoupled Client Integration

In app/inference.py, I refactored predict(raw_bytes) into an HTTP client:
1. It reads the public endpoint dynamically from the API_URL environment variable (e.g. https://23i0703-a2-ops-git-main-h-7bf8.vercel.app/api/predict).
2. It sends an HTTP POST request with the image bytes via requests.post() with a 45-second timeout.
3. It parses the JSON response, returning (label, confidence, raw_probability) to the Flask route.
4. The Flask route logs the transaction to the local PostgreSQL database and renders the verdict to the user.

I updated docker-compose.yml and .env to pass API_URL into the web container. This decoupled architecture allows the user-facing web container to remain lightweight while offloading heavy tensor mathematics to the cloud.

Chapter 9: Conclusion and Future Work

The VisionGuard assignment provided comprehensive experience across the complete machine learning engineering lifecycle: auditing and versioning data with DVC; tracking experiments and registering models with MLflow on DagsHub; overcoming small-sample convergence failures via MobileNetV2 transfer learning; building a defensive Flask application with PostgreSQL audit logging; containerizing services with Docker Compose; and deploying a serverless cloud microservice on Vercel.

Key Engineering Takeaways

1. **Pretrained Foundations Are Essential for Small Data:** Training deep CNNs from scratch on ~1,000 images is ineffective. Pretrained representations provide the necessary inductive bias for rapid convergence.
2. **Decision Thresholds Must Reflect Risk:** Default 0.5 thresholds rarely match business objectives. Calibrating the decision threshold to 0.15 achieved 96.3% recall of synthetic media, minimizing critical false negatives.
3. **Decoupled Microservices Enhance Scalability:** Separating user-facing web servers from serverless inference workers produces lightweight, easily maintainable distributed architectures.

Future Enhancements

• **TFLite & ONNX Quantization:** Converting the Keras model to 8-bit quantized format will reduce model size from 19.3 MB to ~4 MB and reduce serverless cold-start latency.
• **Multi-Generator Attribution:** Expanding the classification objective from binary detection to multi-class attribution, identifying specific generative models (Midjourney, DALL-E, Stable Diffusion).
• **Visual Explainability:** Integrating real-time Grad-CAM heatmaps directly into the web UI to visually highlight regions that contributed to the classification verdict.